

Energy-Efficient Machine Learning-Assisted Load Balancing for Fully Virtualized Access Networks on Multi-Node GPU Servers

Takahiro Suzuki⁽¹⁾, Yasuhiro Matsumoto⁽²⁾, Jun-ichi Kani⁽¹⁾, Tatsuya Shimada⁽¹⁾

⁽¹⁾ Access Network Service Systems Laboratories, NTT, Inc., tkhr.suzuki@ntt.com

⁽²⁾ Center for Information Infrastructure, Institute of Science Tokyo

Abstract We propose energy-efficient machine learning-assisted load balancing for fully-virtualized OLTs. The nonlinear change in computational load is estimated using a decision tree. Experiments on multi-node GPU servers show a 62% improvement in inference error and power savings of 95% to 98%. ©2026 The Author(s)

Introduction

The rapid diversification of service requirements has accelerated the shift from vertically integrated, proprietary network equipment toward disaggregated architectures built on general-purpose computing platforms. Conventional systems, characterized by long development cycles and high capital expenditure, are increasingly being replaced by software implementations that leverage commercial off-the-shelf servers by using software-defined network (SDN) and network function virtualization (NFV) technologies [1, 2]. In the domain of optical access networks, the proposal that re-architected the central office as a data center (CORD) stimulated a paradigm change towards virtualized and programmable infrastructures [3]. Standardization bodies and operator-led consortium have promoted frameworks such as cloud central office (CO) [4] and SDN-enabled broadband access (SEBA) [5]. These initiatives aim to decouple hardware from software, thereby shortening service rollout time, enhancing architectural flexibility, and enabling unified control and management across heterogeneous passive optical network (PON) platforms. To maximize these advantages, progress in expanding the software domain within access systems has been demonstrated in several studies. Examples include software implementations of dynamic bandwidth allocation (DBA) for multi-tenancy of optical line terminals (OLTs) [6], as well as 10G-EPON physical coding sublayer (PCS) processing realized on general-purpose graphic processing unit (GPU) servers [7]. Furthermore, real-time processing of 10 Gb/s $\times$ 4 ports and latency of less than 1 ms have been achieved on GPU servers [8, 9]. For this round-robin (RR) OLT allocation, our previous papers evaluated the power consumption when running a large number of OLTs on a multi-node GPU server as well as a load balancing method responsive to OLT load [10, 11]. However, as this was a initial evaluation, this load balancing method linearly approximated the OLT load based on traffic

volume without distinguishing between upstream and downstream traffic, and did not consider transmission quality parameters such as bit error rate (BER), which have a non-linear impact on processing load. For practical load balancing, accurate allocation of OLTs to GPUs taking these factors into consideration is essential.

In this paper, we propose a novel machine learning-assisted load balancing method that uses pre-training on measured loads to infer OLT loads and utilizes the estimated loads for OLT-GPU allocation while taking into account three parameters downstream traffic, upstream traffic, and BER. Load balancing based on the estimated OLT loads is evaluated on multi-node GPU servers, and the effectiveness of the proposed method in reducing power consumption compared to the conventional RR method of allocating OLTs is evaluated under real-world traffic profiles.

Proposed load balancer

Figure 1 illustrates the proposed machine learning-assisted framework for OLT load estimation and GPU allocation. The framework consists of two phases: an offline training phase and an online inference phase.

In the training phase, the OLT simulator has the ability to generate traffic according to each input parameter and execute 10G-EPON MAC processing, PCS processing, while measuring processing time for the upstream and downstream traffic. The OLT simulator generates traffic under various downstream rates $R^{\text{ds}} = \{r_1^{\text{ds}}, r_2^{\text{ds}}, \dots, r_{N_t}^{\text{ds}}\}$, upstream rates $R^{\text{us}} = \{r_1^{\text{us}}, r_2^{\text{us}}, \dots, r_{N_t}^{\text{us}}\}$, and transmission quality conditions $Q^{\text{us}} = \{q_1^{\text{us}}, q_2^{\text{us}}, \dots, q_{N_t}^{\text{us}}\}$, where N_t is the number of training data points. The corresponding processing times T^{ds} and T^{us} are measured, and the upstream and downstream processing times are summed to get the processing time per OLT by $T = T^{\text{ds}} + T^{\text{us}} = \{t_1, t_2, \dots, t_{N_t}\}$. Since the processing time includes fixed processing time T^{fixed} that does not depend on the amount of calculation, the calculation load $W = \{\omega_1, \omega_2, \dots, \omega_{N_t}\}$

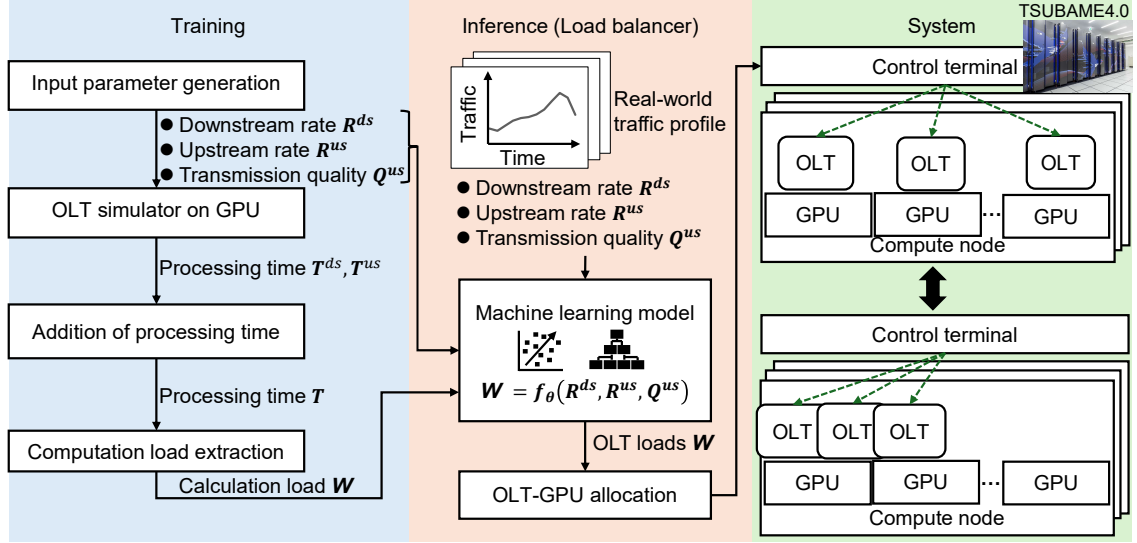


Fig. 1: Proposed load balancer, training method (system configuration tested).

is calculated by subtracting it using equation $W = T - T^{\text{fixed}}$, where T^{fixed} is replaced by the processing time measured when $R^{\text{ds}}, R^{\text{us}}, Q^{\text{us}}$ are sufficiently small values. These parameters are used to train a machine learning model that learns the mapping $W = f_{\theta}(R^{\text{ds}}, R^{\text{us}}, Q^{\text{us}})$, where θ denotes the model parameters. In this study, two machine learning approaches are investigated: linear regression and decision tree. While linear regression is lightweight and models the relationship between input parameters and OLT loads using linear predictor functions, decision tree is an algorithm that captures nonlinear relationships by recursively partitioning the input space into regions with similar OLT loads.

In the inference phase, to preclude directly measuring the processing time, the trained model estimates the OLT computation load from observed traffic parameters for each OLT, $R^{\text{ds}} = \{r_1^{\text{ds}}, r_2^{\text{ds}}, \dots, r_{N_o}^{\text{ds}}\}$, $R^{\text{us}} = \{r_1^{\text{us}}, r_2^{\text{us}}, \dots, r_{N_o}^{\text{us}}\}$, and $Q^{\text{us}} = \{q_1^{\text{us}}, q_2^{\text{us}}, \dots, q_{N_o}^{\text{us}}\}$, where N_o is the number of OLTs. The estimated load, $W = \{\omega_1, \omega_2, \dots, \omega_{N_o}\}$, is provided to OLT-GPU allocation, which allocates OLTs to appropriate GPUs across compute nodes according to the OLT loads. The OLT-GPU allocation problem is modeled as a bin-packing problem similar to the previous work [11]. Each OLT instance is regarded as an item whose weight corresponds to its computation load, while each GPU process is treated as a bin with a fixed processing capacity. Processing capacity is experimentally determined from the load of OLTs that can be implemented within a single GPU without compromising throughput performance. The objective is to minimize the number of active GPU processes. Each OLT must be assigned to exactly one GPU process, and the total computation load allocated to a process must not exceed its capacity, which is experimentally determined. This

problem is efficiently solved by a heuristic first-fit (FF) algorithm. By explicitly modeling the computational load with direction and transmission quality awareness, the proposed framework enables computation-aware resource allocation within software OLT clusters.

Evaluation

The system part of Figure 1 illustrates the experimental configuration used in this study. The experiments were conducted on the TSUBAME4.0 supercomputer at the Institute of Science Tokyo. At the control layer, a control terminal executes the machine learning model and performs OLT-GPU allocation based on the input parameters. The control terminal determines the allocation of OLT instances to GPUs and collects execution logs for GPU power consumption evaluation. Each compute node is equipped with two AMD EPYC 9654 CPUs and four NVIDIA H100 SXM5 GPUs. Within a node, OLT MAC functions are executed on the CPUs, while OLT PCS functions are offloaded to the GPUs. Traffic transmission and reception (Tx/Rx) functions are placed above the MAC layer to emulate OLT processing for each traffic type. In the experiments, up to 50 nodes with 200 GPUs were used simultaneously, with half assigned to OLT processing and the remaining half to ONU processing. Power consumption was measured using a command-line utility provided by NVIDIA.

Figure 2 shows the estimated processing time of linear regression and decision tree together with the measured values when changing the transmission quality expressed as $-\log_{10}(\text{BER})$ for three upstream traffic rates (8.7 Gb/s, 2.1 Gb/s, and 0.54 Gb/s). The results indicate that the processing time decreases as the transmission quality improves (i.e., as $-\log_{10}(\text{BER})$ increases). While linear regression captures the overall decreasing

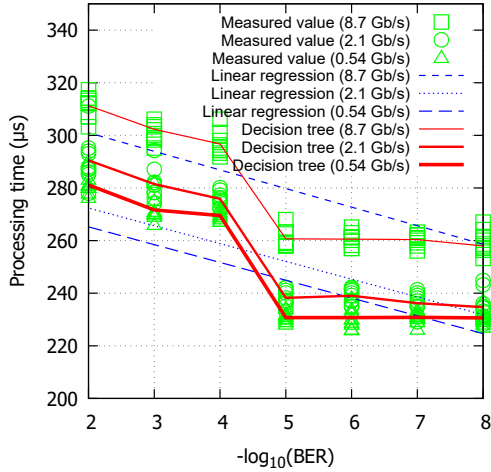


Fig. 2: Estimated processing time of linear regression and decision tree for each traffic rate.

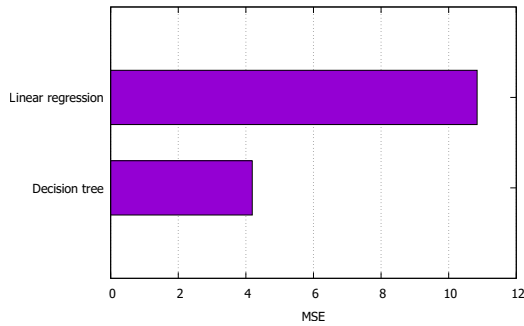


Fig. 3: MSE of linear regression and decision tree.

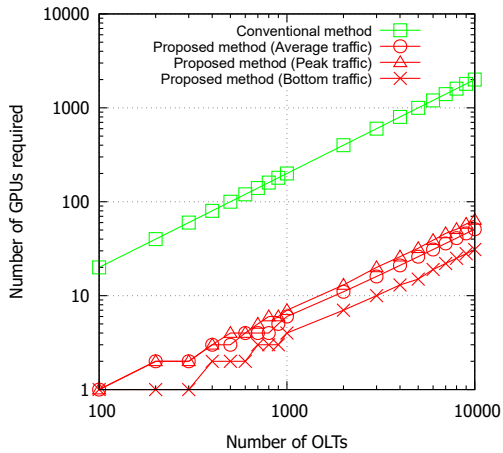


Fig. 4: Number of GPUs required for each number of OLTs.

trend, the decision tree model more accurately follows the nonlinear behavior observed around $-\log_{10}(\text{BER})=5$. Figure 3 shows the mean squared error (MSE) results for linear regression and decision tree inferencing. The decision tree achieves a 62% improvement in error because it can track the nonlinear processing time during estimation.

Figure 4 plots the number of required GPUs as a function of the number of OLTs. The conventional static RR allocation method, which allocates five OLTs to one GPU, is compared with the proposed dynamic FF method based on load estimation using the decision tree under three traffic conditions (downstream, upstream): an average traffic of

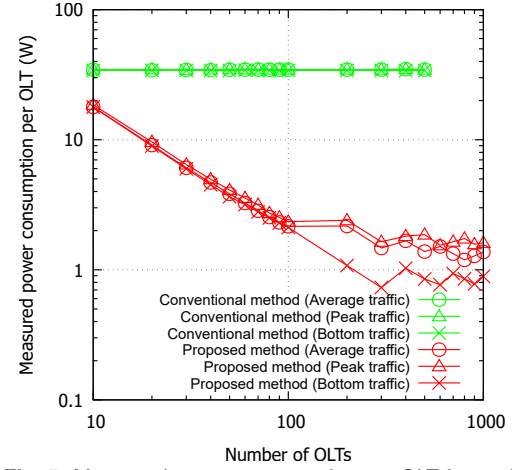


Fig. 5: Measured power consumption per OLT for each number of OLTs.

(112, 12.3) Mb/s, a peak traffic of (183, 16.7) Mb/s, and a bottom traffic of (58.1, 8.56) Mb/s, which are determined from the daily volume of real-world commercial traffic in Japan. While the conventional method starts packing 100 OLTs into 20 GPUs, the proposed method can pack more than 100 OLTs into one GPU across all traffic conditions, and then linearly aggregates OLTs into GPUs from there. With 1,000 OLTs, the required number of GPUs is reduced by 97% to 98%. This confirms that dynamic allocation based on traffic-aware prediction enables efficient resource consolidation. Figure 5 presents the measured GPU power consumption per OLT as a function of the number of OLTs. We measured the power consumption when aggregating OLTs onto GPUs, while confirming that there was no degradation in throughput performance. The conventional method maintains nearly constant power consumption regardless of system scale, indicating limited resource sharing. In contrast, the proposed method achieves substantial power reduction as the number of OLTs increases. The per-OLT power consumption is reduced to nearly 1 W due to improved GPU utilization and workload consolidation. These results confirm that the proposed dynamic GPU allocation method improves energy efficiency by 95% to 98% when the number of OLTs is 100 or more.

Conclusions

We presented a machine learning-assisted GPU allocation framework for fully-virtualized OLTs. By estimating the non-linear computational load for different traffic rates and transmission qualities, the proposed method enables computation-aware resource consolidation across GPU servers. Experiments, conducted on TSUBAME4.0, showed that the decision tree model and dynamic allocation effectively captured nonlinear load characteristics, resulting in a 62% reduction in estimation error and energy savings of up to 98%.

Acknowledgements

This study was carried out using the TSUBAME4.0 supercomputer at the Institute of Science Tokyo.

References

- [1] N. McKeown et al., "Openflow: Enabling innovation in campus networks", *SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–7, 2016. DOI: 10.1145/1355734.1355746.
- [2] ETSI, "Network functions virtualisation - introductory white paper", 2008. [Online]. Available: https://portal.etsi.org/NFV/NFV_White_Paper.pdf.
- [3] L. Peterson et al., "Central office re-architected as a data center", *IEEE Communications Magazine*, vol. 54, no. 10, pp. 96–101, 2016. DOI: MCOM.2016.7588276.
- [4] BBF, "Tr-384 cloud central office reference architectural framework", *CloudCO Technical Reports*, 2018.
- [5] S. Das, "From CORD to SDN enabled broadband access (seba)", *J. Opt. Commun. Netw.*, vol. 13, A88–A99, 2021. DOI: 10.1364/JOCN.402153.
- [6] D. R. Mafioletti, F. Slyne, R. Giller, M. O'Hanlon, D. Coyle, and B. Ryan, "Demonstration of a low latency bandwidth allocation mechanism for mission critical applications in virtual pons with P4 programmable hardware", *2022 Optical Fiber Communications Conference and Exhibition (OFC)*, 2022. DOI: 10.1364/OFC.2022.M3Z.7.
- [7] T. Suzuki, S.-Y. Kim, J. Kani, and J. Terada, "Demonstration of fully softwarized 10G-EPON PHY processing on a general-purpose server for flexible access systems", *Journal of Lightwave Technology*, vol. 38, no. 4, pp. 777–783, 2020. DOI: 10.1109/JLT.2019.2948333.
- [8] T. Suzuki, S.-Y. Kim, J.-i. Kani, and T. Yoshida, "Full softwarization in optical access networks [invited]", *IEEE Access*, vol. 17, no. 7, pp. C62–C69, 2025. DOI: 10.1364/JOCN.549703.
- [9] T. Suzuki, S.-Y. Kim, J.-i. Kani, and T. Yoshida, "Low-latency PON PHY implementation on GPUs for fully software-defined access networks", *IEEE Network*, vol. 36, no. 2, pp. 108–114, 2022. DOI: 10.1109/MNET.002.2100538.
- [10] T. Suzuki, Y. Matsumoto, S.-Y. Kim, J.-i. Kani, and T. Shimada, "Softwarization of 320 10G-EPON OLTs serving 40,960 ONUs with total 2.78-tb/s throughput for fully virtualized central offices", *ECOC*, 2025. DOI: 10.1109/ECOC66593.2025.11262935.
- [11] T. Suzuki, Y. Matsumoto, S.-Y. Kim, J.-i. Kani, and T. Shimada, "Energy-aware load balancing of massive 10G-EPON OLTs on multi-node GPU servers for fully virtualized central offices", *GLOBECOM*, 2025. DOI: 10.1109/GLOBECOM59602.2025.11431698.